



07차시. 반복문

● 학습목표

- 반복문의 개념을 이해하고, 반복적인 작업을 수행하는 데 사용되는 반복문의 역할을 파악할 수 있다.
- for문의 사용 방법을 이해하고, 주어진 횟수만큼 반복 작업을 수행할 수 있다.
- while문의 사용 방법을 이해하고, 조건을 만족하는 동안 반복 작업을 수행할 수 있다.
- do-while문의 사용 방법을 이해하고, 조건을 검사하기 전에 최소 한 번 반복 작업을 수행할 수 있다.
- break와 continue의 사용 방법을 이해하고, 반복문 내에서 제어 흐름을 조절할 수 있다.

7.1 반복문의 개념

반복문(Loop)은 프로그래밍에서 특정 조건이 만족되는 동안 일련의 코드를 반복적으로 실행하기 위한 구조입니다. 반복문을 사용하면 동일한 코드를 여러 번 작성하지 않고도 원하는 횟수만큼 코드를 실행할 수 있습니다.

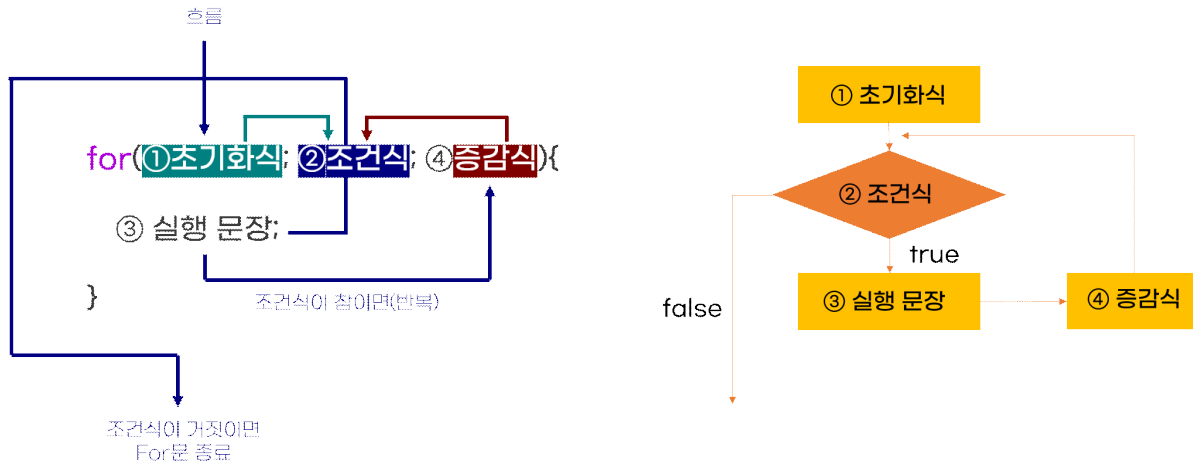
C언어에서는 주로 다음 세 가지 반복문을 사용합니다.

반복문	내용
for문	일정한 횟수만큼 반복을 실행할 때 사용합니다. 초기화, 조건식, 반복 후 처리를 한 줄에 기술할 수 있어 가독성이 높습니다.
while문	조건식이 참인 동안 반복을 실행합니다. 조건식을 만족하면 계속해서 반복을 실행하며, 조건식이 거짓이 되면 반복을 중단합니다.
do-while문	조건식을 검사하기 전에 블록 내의 코드를 최소 한 번은 실행한 후, 조건식이 참인 동안 반복을 실행합니다. 최소한 한 번의 실행을 보장하며, 조건식이 거짓이 되면 반복을 중단합니다.

반복문을 사용할 때 주의할 점은 무한 루프(Infinite Loop)입니다. 무한 루프는 조건식이 항상 참으로 평가되어 프로그램이 끝없이 실행되는 현상입니다. 이를 방지하기 위해 적절한 종료 조건을 설정하거나, 반복 횟수를 제한하는 것이 중요합니다.

7.2 for문

for문은 C언어에서 일정한 횟수만큼 반복을 실행하기 위한 반복문입니다. for문은 초기화, 조건 검사, 반복 후 처리를 하나의 문장에 포함시켜 가독성과 편리성을 높여주는 특징이 있습니다.



[그림 7-1 for문의 순서도]

for문의 구조는 다음과 같습니다.

for문의 기본적인 형태

```
for(초기화; 조건식; 반복 후 처리) {
    // 반복 실행할 코드
}
```

- **초기화**: for문이 처음 실행될 때 한 번만 실행되는 부분입니다. 이곳에서 주로 반복문을 제어하는 변수를 초기화합니다.
- **조건식**: 조건식이 참인 경우에만 반복 블록 내의 코드가 실행됩니다. 조건식이 거짓이 되면 반복문이 종료됩니다.
- **반복 실행할 코드**: 조건식이 참일 때 실행되는 코드 블록입니다.
- **반복 후 처리**: 코드 블록이 실행된 후 반복 후 처리 부분이 실행됩니다. 여기서 주로 반복문을 제어하는 변수의 값을 변경합니다.

이렇게 for문을 사용하면 일정한 횟수만큼 또는 조건에 따라 코드를 반복 실행할 수 있으며, 코드의 간결성과 가독성을 높일 수 있습니다.

다음은 1부터 5까지의 정수를 출력하는 예제입니다.

[예제 7-1] for문 활용1

```
#include <stdio.h>
int main() {
    for(int i =1; i <=5; i++) {
        printf("%d ", i);
    }
    return 0;
}
```

결과 화면

1 2 3 4 5

다음은 1부터 10까지의 정수 중 짝수만 출력하는 예제입니다.

[예제 7-2] for문 활용2

```
#include <stdio.h>
int main() {
    for(int i = 1; i <= 10; i++) {
        if(i % 2 == 0) {
            printf("%d ", i);
        }
    }
    return 0;
}
```

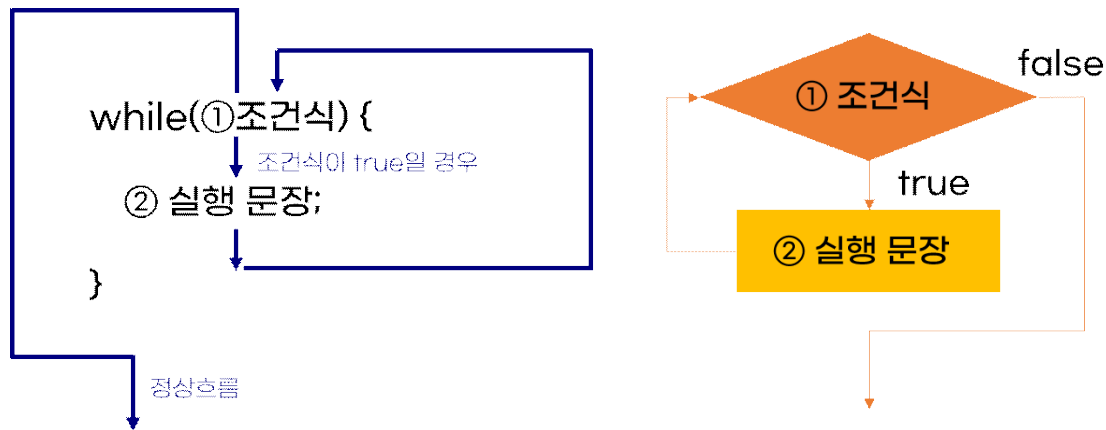
결과 화면

2 4 6 8 10

for문은 for문 하나로 단독으로도 사용할 수 있지만 위의 예제처럼 조건문과 함께 사용할 수도 있습니다.

7.3 while문

while문은 조건이 참인 동안 코드 블록을 반복해서 실행하는 데 사용되는 반복문입니다. 조건이 거짓이 될 때까지 코드 블록 내의 명령문들이 계속 실행됩니다. while문은 주어진 조건을 확인한 후에 조건이 참일 경우 코드 블록을 실행하고, 조건이 거짓이면 코드 블록을 건너뜁니다.



[그림 7-2 while문의 순서도]

while문의 기본 구조는 다음과 같습니다.

while문의 기본적인 형태

```
while (조건) {
    // 조건이 참인 동안 실행할 코드
}
```

while문은 다음과 같은 순서로 작동합니다:

1. 조건을 평가합니다.
2. 조건이 참이면 코드 블록을 실행합니다.
3. 코드 블록의 실행이 끝나면 다시 1단계로 돌아가서 조건을 평가합니다.

주의할 점은 while문 내에서 반복을 제어하는 변수의 값을 변경해야 합니다. 그렇지 않으면 무한 루프에 빠질 수 있습니다.

다음은 1부터 10까지의 합을 구하는 예제입니다.

[예제 7-3] while문 활용1

```
#include <stdio.h>

int main() {
    int sum = 0;
    int i = 1;
    while (i <= 10) {
        sum += i;
        i++;
    }
    printf("1부터 10까지의 합은: %d\n", sum);

    return 0;
}
```

결과 화면

1부터 10까지의 합은: 55

다음은 1부터 20까지의 짝수만 출력하는 예제입니다.

[예제 7-4] while문 활용2

```
#include <stdio.h>

int main() {
    int i = 1;
    while (i <= 20) {
        if (i % 2 == 0) {
            printf("%d ", i);
        }
        i++;
    }

    return 0;
}
```

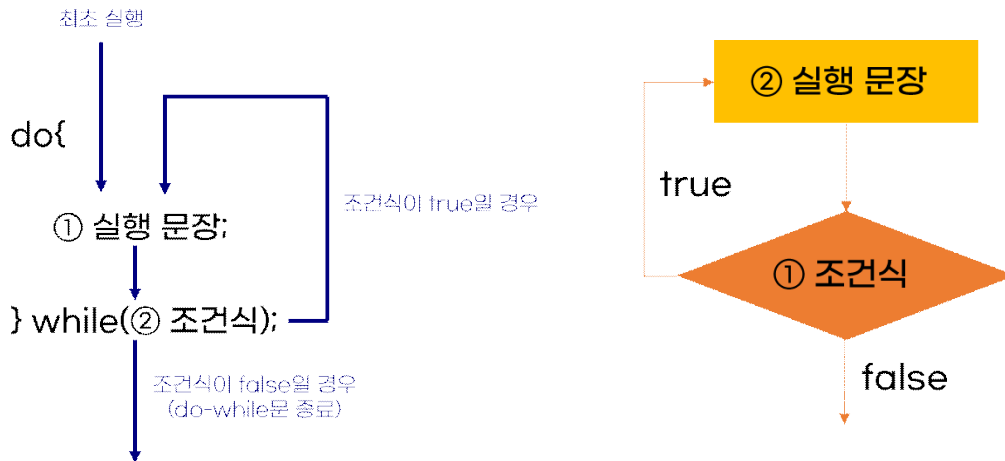
결과 화면

2 4 6 8 10 12 14 16 18 20

while문은 while문 하나로 단독으로도 사용할 수 있지만 위의 예제처럼 조건문과 함께 사용할 수도 있습니다.

7.4 do-while문

do-while문은 조건이 참인 동안 블록 내의 코드를 반복적으로 실행하는 제어문입니다. 단, do-while문은 루프의 마지막에 조건을 확인하기 때문에 블록 내의 코드가 최소 한 번은 실행됩니다. 이 점이 while문과의 주요 차이점입니다.



[그림 7-3 do-while문의 순서도]

do-while문의 구조는 다음과 같습니다.

do-while문의 기본적인 형태

```
do {
    // 코드 블록
} while (조건);
```

do-while문은 다음과 같은 상황에 사용하기에 적합합니다.

1. 조건이 참인지 여부를 확인하기 전에 최소 한 번은 블록 내의 코드를 실행해야 하는 경우.
2. 반복 횟수를 미리 알 수 없는 경우에도 코드를 최소 한 번은 실행하고 싶은 경우.

조건이 처음부터 거짓이더라도 do-while문 내의 코드는 한 번 실행되며, 이후 조건이 참이 될 때까지 반복합니다.

다음은 사용자가 1부터 10까지의 정수를 입력할 때까지 계속 입력을 받는 예제입니다.

[예제 7-5] do-while문 활용1

```
#include <stdio.h>

int main() {
    int number;
    do {
        printf("1부터 10까지의 정수를 입력하세요: ");
        scanf("%d", &number);
    } while (number < 1 || number > 10);
    printf("입력된 정수: %d\n", number);

    return 0;
}
```

결과 화면

사용자가 1부터 10까지의 정수를 입력하면 해당 정수가 출력됩니다.

다음은 사용자가 입력한 숫자를 계속 더하다가, 음수가 입력되면 합계를 출력하는 예제입니다.

[예제 7-6] do-while문 활용2

```
#include <stdio.h>

int main() {
    int input, sum = 0;
    do {
        printf("더할 숫자를 입력하세요 (음수를 입력하면 종료): ");
        scanf("%d", &input);
        if (input >= 0) {
            sum += input;
        }
    } while (input >= 0);
    printf("합계: %d\n", sum);

    return 0;
}
```

결과 화면

사용자가 입력한 양수들의 합계가 출력됩니다. (음수를 입력할 때까지)

do while문은 루프 내의 코드가 먼저 실행되고 나서 조건을 검사합니다. 조건이 참인 경우에 루프를 계속 반복하고, 거짓이면 루프를 종료합니다. 일반적으로 사용자 입력, 파일 읽기 등의 작업에서 최소한 한 번은 실행이 보장되어야 할 때 do-while문을 사용합니다.

7.5 break와 continue

break와 continue는 C언어의 반복문에서 흐름을 제어하는데 사용되는 키워드입니다. 이 두 키워드는 for, while, do-while과 같은 반복문에서 사용됩니다.

break문은 현재 실행 중인 반복문을 즉시 종료하고, 반복문의 바깥쪽 코드로 이동합니다. 보통 조건문과 함께 사용되어 특정 조건이 충족되면 루프를 빠져나오고자 할 때 사용됩니다.

break문의 예시

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break;
    }
    printf("%d ", i);
}
```

위 예제에서 i가 5가 되면 break문이 실행되고, 반복문이 즉시 종료됩니다. 따라서 출력 결과는 "0 1 2 3 4"입니다.

continue문은 현재 반복문의 나머지 부분을 건너뛰고, 다음 반복으로 이동합니다. continue문을 만나면 반복문의 조건 검사로 이동하게 되며, 이를 통해 특정 조건에서 루프의 일부분을 실행하지 않을 수 있습니다.

continue문의 예시

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    printf("%d ", i);
}
```

위 예제에서 i가 짝수일 때 continue문이 실행되고, 홀수일 때만 printf문이 실행됩니다. 따라서 출력 결과는 "1 3 5 7 9"입니다.



연습문제

1. 1부터 10까지의 숫자 중 짝수만 출력하는 프로그램을 작성하세요. (for문 사용)

결과 화면

2 4 6 8 10

2. 1부터 10까지의 숫자 중 홀수만 출력하는 프로그램을 작성하세요. (while문 사용)

결과 화면

1 3 5 7 9

3. 사용자로부터 0이 입력될 때까지 숫자를 입력받아 입력된 숫자의 합을 출력하는 프로그램을 작성하세요. (do while문 사용)

결과 화면

(입력된 숫자가 "4 5 2 0"인 경우): "입력된 숫자의 합: 11"



연습문제: 정답

1. 1부터 10까지의 숫자 중 짝수만 출력하는 프로그램을 작성하세요. (for문 활용)

연습문제:정답 7-1

```
#include <stdio.h>
```

```
int main() {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            printf("%d ", i);
        }
    }

    return 0;
}
```

2. 1부터 10까지의 숫자 중 홀수만 출력하는 프로그램을 작성하세요. (while문 활용)

연습문제:정답 7-2

```
#include <stdio.h>
```

```
int main() {
    int i = 1;
    while (i <= 10) {
        if (i % 2 != 0) {
            printf("%d ", i);
        }
        i++;
    }

    return 0;
}
```

3. 사용자로부터 0이 입력될 때까지 숫자를 입력받아 입력된 숫자의 합을 출력하는 프로그램을 작성하세요. (do while문 활용)

연습문제:정답 7-3

```
#include <stdio.h>

int main() {
    int num, sum = 0;
    do {
        printf("숫자를 입력하세요 (0을 입력하면 종료): ");
        scanf("%d", &num);
        sum += num;
    } while (num != 0);
    printf("입력된 숫자의 합: %d\n", sum);

    return 0;
}
```